

ASSIGNMENT 1

BUILDER PATTERN

SoftwareDesign Patterns
Java Implementation: Email Builder

Student: Damir Shuitinov
Group: SE-2532
2026

1. Introduction

Builder creates objects step by step and separates construction from representation. This project uses an Email as the product.

The same EmailBuilder steps create an Email object and a text preview.

2. Builder Pattern Structure

Following Lecture 1, the project uses Product, Builder, Concrete Builder, Director, and Client.

Lecture	component	Class in this project	Responsibility
Product		Email	Stores final email data.
Builder		EmailBuilder	Defines shared steps.
Concrete Builder		EmailObjectBuilder	Builds an Email object.
Concrete Builder		EmailPreviewBuilder	Builds a text preview. Defines
Director		EmailDirector	reusable build sequences. Starts construction and gets results.
Client		Main	

3. UML Class Diagram

EmailDirector uses only EmailBuilder. Each concrete builder has its own getResult() because the result types are different.

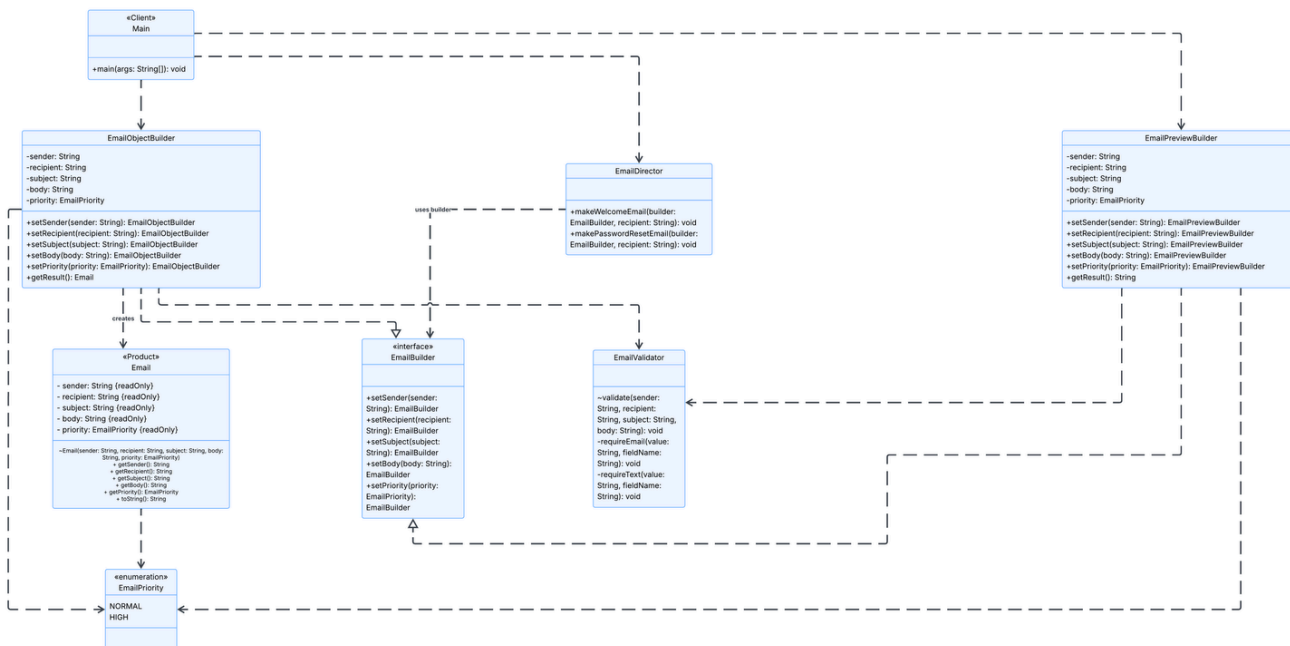


Figure 1. Builder Pattern structure for the Email product

4. Implementation

4.1 Builder interface and fluent API

Each construction step returns the builder, so calls can be chained.

```
public interface EmailBuilder {  
    EmailBuilder setSender(String sender);  
    EmailBuilder setRecipient(String recipient);  
    EmailBuilder setSubject(String subject);  
    EmailBuilder setBody(String body);  
    EmailBuilder setPriority(EmailPriority priority);  
}
```

4.2 Immutable Product

Email is immutable: its fields are final, its constructor has package access, and it has no setters.

```
public final class Email {  
    private final String sender;  
    private final String recipient;  
    private final String subject;  
    private final String body;  
    private final EmailPriority priority;  
  
    Email(String sender, String recipient, String subject,  
          String body, EmailPriority priority) {  
        this.sender = sender; this.recipient =  
        recipient; this.subject = subject;  
        this.body = body; this.priority = priority;  
    }  
}
```

4.3 Two concrete builders

Email Object Builder returns Email; Email Preview Builder returns String. Both use the same construction steps.

```
public Email getResult() {  
    EmailValidator.validate(sender, recipient, subject, body);  
    if (priority == null) {  
        throw new IllegalStateException("Priority is required");  
    }  
    return new Email(sender, recipient, subject, body, priority);  
}  
  
public String getResult() {  
    EmailValidator.validate(sender, recipient, subject, body);  
    if (priority == null) {  
        throw new IllegalStateException("Priority is required");  
    }  
    return formatPreview();  
}
```

4.4 Director with reusable configurations

Email Director has two ready configurations: welcome and password reset. It accepts only EmailBuilder.

```

public void makeWelcomeEmail(EmailBuilder builder, String recipient) {
    builder.setSender(SYSTEM_SENDER)
        .setRecipient(recipient)
        .setSubject(WELCOME_SUBJECT)
        .setBody(WELCOME_BODY)
        .setPriority(EmailPriority.NORMAL);
}

public void makePasswordResetEmail(EmailBuilder builder, String recipient) {
    builder.setSender(SYSTEM_SENDER)
        .setRecipient(recipient)
        .setSubject(RESET_SUBJECT)
        .setBody(RESET_BODY)
        .setPriority(EmailPriority.HIGH);
}

```

5. Clean Code Principles

1. Small functions - do one thing

Each setter stores one value and returns the builder.

```

public EmailPreviewBuilder setSubject(String subject) {
    this.subject = subject;
    return this;
}

```

2. Descriptive and consistent names

Names such as make PasswordResetEmail() and EmailPreviewBuilder clearly show their purpose.

```

public void makePasswordResetEmail(EmailBuilder builder, String recipient) {
    ...
}

```

3. One level of abstraction per function

getResult() stays high-level. Validation and formatting are handled by separate methods.

```

public String getResult() {
    EmailValidator.validate(sender, recipient, subject, body);
    if (priority == null) {
        throw new IllegalStateException("Priority is required");
    }
    return formatPreview();
}

```

4. DRY - do not repeat validation logic

Email Validator keeps shared validation in one place.

```

static void validate(String sender, String recipient,
                    String subject, String body) {
    requireEmail(sender, "Sender");
    requireEmail(recipient, "Recipient");
    requireText(subject, "Subject");
    requireText(body, "Body");
}

```

5. Prefer exceptions over error codes

Invalid input throws Illegal State Exception instead of returning an error value.

```
if (value == null || value.isBlank()) {  
    throw new IllegalStateException(fieldName + " is required");  
}
```

6. Avoid magic strings and values

Director values use named constants, and priority uses an enum.

```
private static final String SYSTEM_SENDER = "no-reply@example.com"; private static final String  
WELCOME_SUBJECT = "Welcome to Builder Demo";  
  
public enum EmailPriority {  
    NORMA  
    L,  
    HIGH  
}
```

6. Conclusion

The project follows the Builder structure from Lecture 1. The same steps create two Email representations, while the Director stays independent of the concrete builder.

7. GitHub Repository

https://github.com/EZDAMIR/JS_Intro/tree/builder-pattern

The project contains incremental commits, as required.